

Isomera v2: Scenario Materialization, Curation, and Benchmark Publication Report

Generated by Isomera v2

2026-04-18T10:47:54

Abstract

This report documents an end-to-end Isomera v2 execution: relational scenario selection, lineage graph materialization, manual validation, supervised dataset creation, model training, and benchmark publication. It is generated as a reproducible package containing the article draft, source data, model artifacts, figures, and metadata.

1 Introduction

Isomera v2 supports redundancy analysis in Data Mesh-like architectures by representing data products and transformations as directed lineage graphs. The system combines graph visualization, candidate-pair filtering, manual ground-truth creation, benchmark execution, and GNN-based model training.

The main reproducible path used by this report is database source selection, relational schema loading, lineage graph materialization, candidate-pair reduction, manual duplicate validation, supervised dataset publication, model training, benchmark execution, and article package export.

2 Related Work Context

The report package is designed to support a later IEEE-style article. It connects graph-based lineage modeling, graph isomorphism, duplicate detection, and graph neural networks into one reproducible experimental workflow.

3 System Architecture

Isomera v2 separates source scenario materialization, curation, model training, benchmark execution, and publication packaging.

The architecture deliberately separates the source benchmark warehouse from the publication backend. PostgreSQL represents the external relational benchmark under analysis. MySQL stores curated publication evidence, reviewed pairs, report summaries, and model references. This split prevents benchmark source data from being mixed with experimental metadata and makes it easier to export a reproducible package.

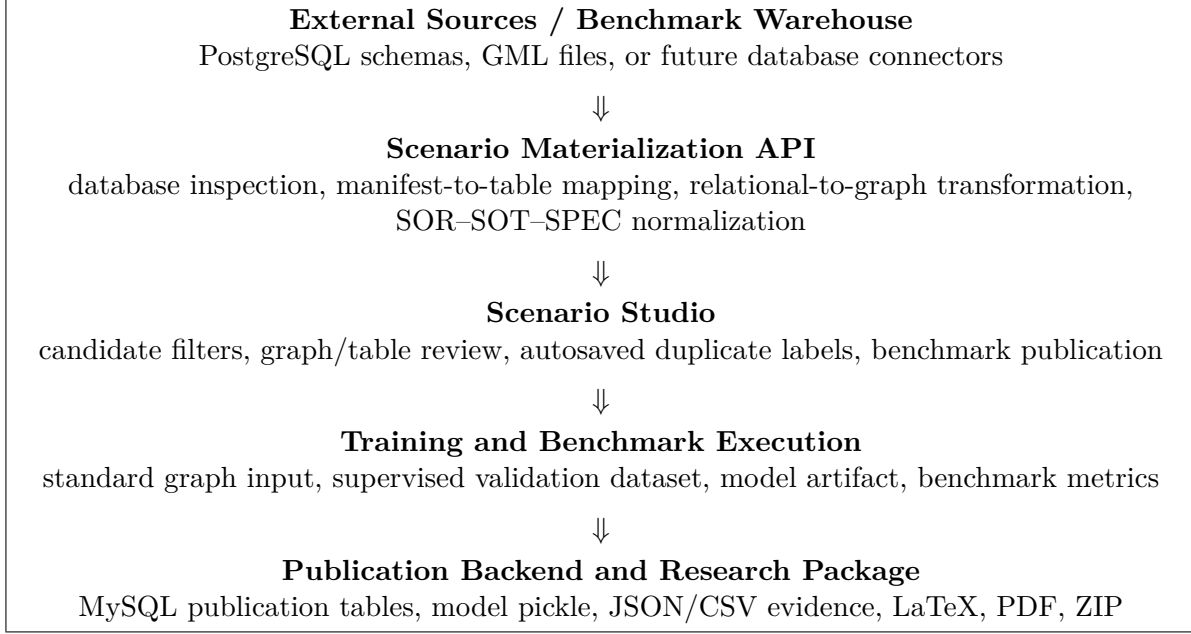


Figure 1: IEEE-style high-level Isomera v2 architecture used in the generated research package.

3.1 Component Responsibilities and Rationale

component	responsibility	rationale
Streamlit UI	Provides the interactive workflow for loading scenarios, reviewing candidate pairs, training models, inspecting stores, and exporting research packages.	Streamlit keeps the research prototype fast to iterate while still exposing every intermediate artifact needed for paper writing.
Scenario Materialization API	Converts PostgreSQL schemas, GML files, or manual graphs into one normalized graph contract.	Centralizes the table-to-graph transformation so UI, training, benchmark, and reports consume the same graph, edge table, adjacency matrix, and metadata.
PostgreSQL Scenario Warehouse	Stores benchmark relational scenarios as database schemas.	Represents the external relational environment being analyzed; it is the source of scenario data, not the operational backend.
MySQL Publication Backend	Stores published benchmarks, scenarios, nodes, edges, reviewed pairs, and publication reports.	Separates article/research evidence from the benchmark warehouse and prepares the project for a durable backend store.
GML/JSON File Layer	Stores portable graphs, labels, manifests, article captures, and exported package metadata.	Provides reproducible files for DOI/data sharing and makes the benchmark portable outside the local database.
Training Pipeline	Transforms normalized graphs and curated duplicate labels into supervised datasets and pickle model artifacts.	Keeps model input standardized so future GNN/CNN/other detectors can share the same upstream materialization contract.
Research Report Builder	Generates LaTeX, PDF, ZIP, figures, CSVs, JSONs, and model references from Article Capture evidence.	Turns experimental actions into article-ready evidence without manually copying app state into the manuscript.

3.2 Architectural Layers

layer	responsibility
Source Warehouse	PostgreSQL/GML scenarios used as reproducible lineage sources.
Scenario Materialization API	Normalizes each scenario into graph, edge table, matrix, and metadata.

Curation and Validation	Produces supervised duplicate-pair datasets with filters and target labels.
Training and Benchmark	Routes detector families and model clusters to each evaluated scenario.
Research Reports	Exports JSON, CSV, figures, LaTeX, PDF, ZIP, and model references.

3.3 Data Stores

store	technology	purpose
PostgreSQL Scenario Warehouse	PostgreSQL	Relational benchmark source schemas.
MySQL Publication Backend	MySQL	Publication/backend evidence, logs, reports, and curated scenario metadata.
GML/JSON	Files	Portable graphs, labels, manifests, and model routing.
Research Packages	LaTeX/PDF/ZIP	Article-ready reproducibility package.

4 Database Architecture

The benchmark warehouse is stored in PostgreSQL. Each benchmark scenario is represented by one schema, such as `scenario_sor2_d5_seed42`. In practical terms, SOR2 means that the scenario was generated with two source-of-record nodes per configured domain group, D5 means that five business domains are present, and seed42 identifies the reproducible random seed/contract variant. If two scenarios are selected, Isomera reads two schemas and produces two normalized graphs, two validation datasets, and then one benchmark-level model/metrics package that references both scenarios.

The MySQL backend is used as the publication/research backend for scenarios, nodes, edges, reviewed pairs, model artifacts, reports, and future operational logs. SQLite is retained only as a local fallback while the backend migration is completed.

4.1 TPC-DS Domain Catalog

The current TPC-DS pilot uses five practical business domains. The D identifiers are not arbitrary visual labels: they identify the domain bucket used to generate SOR, SOT, and SPEC tables and to later filter candidate pairs. Future D6/D7 domains should be added through the same manifest-based contract.

domain	practical_context	examples	extension_rule
D1	Customer, catalog, and warehouse-performance context.	customer, customer__demographics, catalog__performance, warehouse__logistics.	Keep D1 when the business view is customer/catalog performance.
D2	Store, geography, customer attributes, and order summary context.	nation, store, customer_attr, customer_orders, customer_summary.	Use the same pattern for another geography/store/customer domain.
D3	Item, promotion, catalog sales, and analytical reuse context.	item, promotion, catalog_sales, time_analysis.	Use for product/promotion/catalog-sales analytical scenarios.
D4	Date, time, customer orders, and warehouse stock context.	date_dim, time_dim, customer_orders, warehouse_stock.	Use for temporal and inventory-history scenarios.
D5	Income band, warehouse, store sales, and store-sales summary context.	income_band, warehouse, store_sales, store_sales_summary.	Use for income/warehouse/store-sales scenarios.

D6+	Future user-defined domain.	Any new business domain introduced by the user.	Add a manifest/mapping contract defining domain name, SOR tables, SOT tables, SPEC outputs, and lineage edges.
-----	-----------------------------	---	--

4.2 Source Details

field	value
benchmark	tpc_ds_default
display_name	TPC-DS (Default)
architecture_root	/Users/cayofel/Documents/GitHub/isomera_v2/main/data/architectures/tpc_ds
candidate_scope	all
runs	10
best_of_all	True

4.3 Selected Benchmark Scenarios

When more than one scenario is selected, each PostgreSQL schema is materialized independently and then grouped under the same benchmark-level execution. The full scenario table is exported as CSV inside the package.

scenario	schema	nodes	edges	positive_pairs
graph_SOR2_D1_seed42	scenario_sor2_d1_seed42	6	6	1
graph_SOR2_D2_seed42	scenario_sor2_d2_seed42	12	13	3
graph_SOR2_D3_seed42	scenario_sor2_d3_seed42	18	19	5
graph_SOR2_D4_seed42	scenario_sor2_d4_seed42	24	24	7
graph_SOR2_D5_seed42	scenario_sor2_d5_seed42	30	32	17
graph_SOR4_D1_seed42	scenario_sor4_d1_seed42	12	14	8
graph_SOR4_D2_seed42	scenario_sor4_d2_seed42	24	32	23
graph_SOR4_D3_seed42	scenario_sor4_d3_seed42	36	42	44
graph_SOR4_D4_seed42	scenario_sor4_d4_seed42	48	63	75
graph_SOR4_D5_seed42	scenario_sor4_d5_seed42	60	70	116
graph_SOR8_D1_seed42	scenario_sor8_d1_seed42	21	30	23
graph_SOR8_D2_seed42	scenario_sor8_d2_seed42	42	61	62
graph_SOR8_D3_seed42	scenario_sor8_d3_seed42	63	92	142
graph_SOR8_D4_seed42	scenario_sor8_d4_seed42	84	122	246
graph_SOR8_D5_seed42	scenario_sor8_d5_seed42	105	153	376
graph_SOR16_D1_seed42	scenario_sor16_d1_seed42	29	30	17

graph_SOR16_ D2_seed42	scenario_sor16_d2_ seed42	58	61	45
graph_SOR16_ D3_seed42	scenario_sor16_d3_ seed42	87	92	102
graph_SOR16_ D4_seed42	scenario_sor16_d4_ seed42	116	122	192
graph_SOR16_ D5_seed42	scenario_sor16_d5_ seed42	145	153	290

4.4 MySQL Publication Backend Validation

No rows available.

5 Scenario Materialization API

Expose a reproducible source-to-graph transformation contract used by the UI, training code, benchmark runner, and report builder.

Technically, the API is the reproducibility boundary of Isomera. Every supported source must be transformed into the same materialized contract before visualization, validation, training, benchmarking, or reporting. This avoids having one representation for the UI and another for the model.

5.1 API Functions

function	input	output
load_source	benchmark architecture or database schema	scenario graph and labels
normalize_lineage	SOR/SOT/SPEC node metadata and edges	directed graph normalized to SOR -> SOT -> SPEC when semantics are available
candidate_scope	all, SPEC, or SOT+SPEC	evaluated candidate-pair universe
route_models	scenario name and detector cluster	scenario-specific pickle path or post-hoc best-of selector

5.2 API Input/Output Contract

artifact	description	consumer
normalized_graph	NetworkX directed graph with canonical edge direction when layer semantics are available.	Scenario Studio, graph visualization, candidate filtering, training.
lineage_structure	Node table with domain, layer, node, source table, semantic name, in-degree, and out-degree.	Research Reports, Admin inspection, article tables.
edge_table	Directed edge table with origin, destination, and edge type.	Report package, DOI export, future semantic edge extensions.
adjacency_matrix	Binary adjacency matrix aligned to the normalized graph.	Graph diagnostics, report figures, future CNN/image-based detector experiments.
source_metadata	Database URL, schema, build mode, manifest usage, manifest path, table count, and graph build steps.	Article Capture, package manifest, reproducibility section.

5.3 Normalization Method

The current normalization is not a general semantic inference engine. It is a deterministic layer-aware graph-direction procedure. It works reliably when the source exposes SOR, SOT, and SPEC

semantics through names, node attributes, or a manifest. For arbitrary databases, Isomera needs a mapping contract before it can claim semantic SOR-SOT-SPEC normalization.

step	technical_rule	limitation
Layer detection	Each node is assigned a rank by inspecting node names and metadata: SOR=0, SOT=1, SPEC=2, OTHER=3.	If a database uses arbitrary names such as customers, orders, mart_sales without a manifest or mapping, the API cannot know which layer each table belongs to.
Direction scoring	For every edge, the API compares source_rank and target_rank. Edges with source_rank < target_rank count as downstream; source_rank > target_rank count as upstream.	The score only measures consistency with known layer ranks. It does not infer business semantics by itself.
Graph reversal	If upstream_edges_before is greater than downstream_edges_before, all edges are reversed. Otherwise the graph is copied as-is.	Mixed or ambiguous graphs can be partially incorrect if the source does not expose layer semantics or a reliable manifest.
Manifest mode	When a benchmark manifest is available, it provides node, table_name, semantic_name, type/layer, domain, and explicit lineage edges.	This is the preferred article-grade mode, but it requires preparing or importing a mapping contract.
Foreign-key-only mode	Without a manifest, the API can inspect tables and foreign keys to build a graph, then apply the same direction heuristic.	Foreign keys are physical constraints, not necessarily transformation lineage. This mode is exploratory unless validated by the user.

5.4 API Limits

limit
Generic databases without a manifest or SOR/SOT/SPEC semantics cannot be normalized semantically; they require a mapping contract.
Best-of-all routing is a diagnostic selector and must be reported as post-hoc/oracle-style evidence unless the selection policy is fixed before execution.
Accuracy is retained but SF-Jaccard is the primary metric under pair-imbalance.

6 Pipeline

The pipeline is source $\rightarrow$ normalized graph $\rightarrow$ validation dataset $\rightarrow$ training dataset $\rightarrow$ model artifact.

stage	input	output	details
source	tpc_ds	20 GML scenarios	Original TPC-DS benchmark using the default curated duplicate pairs.
normalized graph	GML + node layer metadata	NetworkX directed graph	Edges and nodes are evaluated under the selected candidate scope.
validation dataset	real_pairs JSON	1794 positives inside 39695 candidates	Pairs outside the selected candidate scope are excluded from metric denominators.
model routing	scenario name + cluster map	pickle path per scenario	Best-of-all is post-hoc diagnostic routing when enabled.
benchmark execution	detector families	10 family rows	10 runs per detector family per scenario.

7 Lineage Graph and Matrix Views

The normalized graph contains 1020 nodes and 1231 directed edges. The direction is normalized to SOR $\rightarrow$ SOT $\rightarrow$ SPEC whenever the manifest or table names expose those layers.

8 Candidate Filtering and Ground Truth

The final supervised table is produced by selecting candidate filters, reviewing each pair, and assigning a binary target. The target is 1 for duplicate and 0 for not duplicate. This table is the ground truth used by the training pipeline. A small subset can be used only to smoke-test the MySQL store, but article-grade datasets must come from complete ground-truth review or a documented benchmark label source such as TPC-DS `real_pairs`.

quantity	value
total_pairs	39695
candidate_pairs	39695
reviewed_pairs	39695
duplicate_pairs	1794

8.1 Filter Protocol

filter	setting	effect
candidate_scope	all	Defines the candidate-pair universe used by metrics.

8.2 Supervised Validation Dataset

No rows available.

9 Training Configuration

The current model family is the Graph Isomorphism Network Pair Classifier. The article-grade path uses a supervised validation dataset with `target=1` for duplicate pairs and `target=0` for non-duplicate pairs. Legacy-compatible runs can still derive negatives through negative sampling, but the report always records the effective loss, optimizer, split, and balancing strategy used to train the pickle.

The model input is intentionally standardized: any future connector must first produce the same normalized graph, edge table, adjacency matrix, and supervised pair table. This is what allows a pickle trained from one curated benchmark to be evaluated consistently by Benchmark & Examples.

scenario	positive_pairs	negative_pairs	dataset_rows
graph_SOR2_ D1_seed42	1	14	15
graph_SOR2_ D2_seed42	3	63	66
graph_SOR2_ D3_seed42	5	148	153
graph_SOR2_ D4_seed42	7	269	276
graph_SOR2_ D5_seed42	17	418	435
graph_SOR4_ D1_seed42	8	58	66
graph_SOR4_ D2_seed42	23	253	276

graph_SOR4_ D3_seed42	44	586	630
graph_SOR4_ D4_seed42	75	1053	1128
graph_SOR4_ D5_seed42	116	1654	1770

Only the first 10 rows are shown; the full table is exported as CSV.

9.1 Available Training Options

The following options are exposed in Isomera. The friendly name is what the user sees in the application; the technical function is what the backend actually executes or records.

group	friendly_name	technical_function	formula	use
optimizer	Adaptive gradient optimizer (torch.optim.Adam)	torch.optim.Adam	$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$; $v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$	Default optimizer for the GNN. It adapts the learning rate per parameter using first and second gradient moments.
optimizer	Adam with decoupled weight decay (torch.optim.AdamW)	torch.optim.AdamW	$\theta_t = \theta_{t-1} - \eta (\text{AdamGrad}(\theta_{t-1}) + \lambda \theta_{t-1})$	Adam variant with decoupled weight decay. Useful when later experiments add explicit regularization.
optimizer	Stochastic gradient descent (torch.optim.SGD)	torch.optim.SGD	$\theta_t = \theta_{t-1} - \eta \text{grad}_{\theta} L$	Simple gradient descent baseline. It is slower but useful as a methodological control.
loss	Binary cross entropy on logits (torch.nn.BCEWithLogitsLoss)	torch.nn.BCEWithLogitsLoss	$L = -[y \log(\text{sigmoid}(z)) + (1-y) \log(1-\text{sigmoid}(z))]$	Default binary classification loss. It receives raw logits and applies the sigmoid internally in a numerically stable way.
loss	Weighted binary cross entropy (torch.nn.BCEWithLogitsLoss(pos_weight))	torch.nn.BCEWithLogitsLoss(pos_weight)	$L = -[\text{pos_weight} * y \log(\text{sigmoid}(z)) + (1-y) \log(1-\text{sigmoid}(z))]$	Same BCE loss, but positive duplicate pairs receive a larger weight when the dataset has many more negatives.
loss	Focal loss for rare duplicates (custom sigmoid focal loss)	custom sigmoid focal loss	$FL(p_t) = -\alpha * (1 - p_t)^\gamma * \log(p_t)$	Focuses training on hard examples by down-weighting easy negatives. Useful when almost every pair is non-duplicate.
balancing	Weight duplicate class in the loss (torch.nn.BCEWithLogitsLoss(pos_weight))	torch.nn.BCEWithLogitsLoss(pos_weight=N_negative/N_positive)	$\text{pos_weight} = N_{\text{negative}} / N_{\text{positive}}$	Recommended default for article-grade training. It keeps the supervised dataset intact and increases the penalty for missing rare duplicate pairs.
balancing	Sample negatives by ratio (negative_sampling)	negative_sampling with negative_ratio	$N_{\text{negative_sampled}} = N_{\text{positive}} * \text{negative_ratio}$	Legacy-compatible path. Positives come from curated labels and negatives are sampled from non-labeled graph pairs.
balancing	Use real distribution without balancing (no sampler/no class weight)	no balancing	$D_{\text{train}} = D_{\text{train_original}}$	Keeps the imbalanced distribution exactly as produced by the supervised table. Useful as a baseline, but can hide duplicate failures behind high accuracy.

balancing	Reduce non-duplicates in training (random undersampling)	random undersampling of target=0	$N_negative_kept \leq N_positive * negative_ratio$	Keeps all positives and randomly reduces negatives. Faster, but may discard useful negative diversity.
balancing	Repeat duplicate pairs in training (random oversampling)	random oversampling of target=1	$N_positive_repeated \sim N_negative$	Duplicates positive rows so the model sees rare duplicate evidence more often. Can overfit if positives are too few.
balancing	Balanced training batches (balanced batch sampler)	balanced positive/negative epoch sampler	batch $\sim 50\%$ target=1 + 50% target=0	Approximates balanced batches by oversampling positives before each epoch. This stabilizes gradients in highly imbalanced training.
balancing	Prefer difficult non-duplicates (hard negative mining)	structural hard-negative sampler	$score = nodes_a - nodes_b + edges_a - edges_b $	Keeps negatives whose subgraphs have similar size/edge profiles to positives. These are harder examples than random non-duplicates.

9.2 Strategy Theory

The imbalance strategies below are not cosmetic labels. They change either the loss function or the composition of the training examples. This distinction matters for the article because duplicate pairs are rare and high accuracy can be achieved without discovering duplicates.

strategy	technical_function	formula	interpretation
Weighted BCE	torch.nn.BCEWithLogitsLoss(pos_weight)	$L = -[w_p y \log(\text{sigmoid}(z)) + (1-y) \log(1-\text{sigmoid}(z))]$	Keeps all validated pairs and increases the cost of missing rare duplicate pairs. This is the preferred first strategy when positives are scarce.
Focal Loss	custom sigmoid focal loss	$FL(p_t) = -\alpha(1-p_t)^\gamma \log(p_t)$	Down-weights easy examples and concentrates gradient on hard or misclassified pairs. It is useful when the model otherwise learns only the dominant negative class.
Hard Negative Mining	structural hard-negative sampler + BCEWithLogitsLoss	$score = nodes_a - nodes_b + edges_a - edges_b $	Keeps non-duplicate pairs that look structurally similar to positives. This tests whether the model can separate difficult near-matches instead of only easy negatives.

9.3 Isomera Staged Hyperparameter Protocol

The recommended Isomera protocol is a staged search, not an exhaustive grid. Exhaustive search grows combinatorially and would spend most compute on configurations that are unlikely to survive model selection. The staged protocol first screens representative scenarios, selects the top configurations by SF-Jaccard, and only then runs the complete 20-scenario validation. Users can still bypass this protocol and train a manual configuration when they want a single controlled run.

For the proposed grid, one configuration means one combination of training strategy, learning rate, hidden-channel size, dropout, and inference threshold. The reduced grid has 108 configurations. Across three benchmark variants, screening on five representative scenarios requires 1620 trainings. Final validation of the top five configurations per benchmark over all 20 scenarios adds 300 trainings, for a total of 1920 trainings before the final detector-family benchmark.

parameter	values	count	reason
training strategy	Weighted BCE; Focal Loss; Hard Negatives	3	Compares the main imbalance-handling families used by the GNN pipeline.
learning rate	0.001; 0.005; 0.010	3	Controls optimizer step size and training stability.
hidden channels	16; 32	2	Controls embedding capacity while keeping the search lightweight.
dropout	0.0; 0.1	2	Tests whether regularization improves generalization on scarce positives.
inference threshold	0.4; 0.5; 0.6	3	Moves the decision boundary between conservative and recall-oriented duplicate detection.

stage	scope	trainings	selection_rule	protocol_output
screening_5_scenarios	3 benchmarks x 5 representative scenarios x 108 configs	1620	Rank configurations by SF-Jaccard, then retain top 5 per benchmark.	Documents broad search behavior without paying the cost of a full exhaustive grid.
full_validation_20_scenarios	3 benchmarks x 20 scenarios x top 5 configs	300	Retrain and evaluate only the top configurations across the complete benchmark suite.	Produces final results with complete scenario coverage and controlled compute cost.
benchmark_final	Best GNN configurations vs VF2, Node Match, GNN TPC-DS v1, GNN GenAI v1, and GNN GenAI v2	0	Report detector-family metrics by scenario and aggregate family.	Separates model-selection evidence from final detector-family comparison.

9.4 Options Used in This Run

setting	selected	technical	effect_in_run
optimizer			updates GIN and pair-classifier parameters
loss			pos_weight=, alpha=, gamma=
balancing			
train_distribution		target counts after balancing	validation/test distribution=

9.5 Training Data Flow

step	operation	real_value
1. Normalize graph input	Load the curated scenario graph and normalize direction to SOR -> SOT -> SPEC.	scenarios=[]
2. Build node-centered subgraphs	For each candidate node, extract a local subgraph. SPEC nodes use upstream lineage; SOR nodes use downstream lineage; SOT nodes use both directions.	subgraphs generated from normalized NetworkX graph
3. Encode graph tensors	Convert each subgraph into node feature matrix x and edge_index. Current baseline uses one scalar node feature per node.	x = ones(num_nodes, 1)

4. Sample supervised pairs	Use the supervised validation table when available. In legacy-compatible runs, curated duplicate pairs define positives and negatives are sampled from non-labeled graph pairs.	negative_ratio=, generated_rows=
5. Split train/test	Split the supervised dataset into train and validation/test partitions.	train_ratio=, test_ratio=, train_size=, val_size=
6. Balance training distribution	Apply the selected imbalance strategy only to the training partition. The validation/test partition keeps its observed label distribution.	strategy=, operation=
7. GIN embedding layers	Apply Graph Isomorphism Network aggregation to produce one embedding per subgraph.	hidden_channels=
8. Pair classifier	Concatenate two subgraph embeddings and classify the pair with an MLP binary head.	output logit interpreted through sigmoid for duplicate probability
9. Optimization	Optimize the configured loss with the selected optimizer over the configured epochs.	loss=, epochs=, learning_rate=, dropout=

9.6 Model and Hyperparameters

field	value
model_family	VF2_NodeMatch_GNN_clusters
model_name	all_detector_families
model_path	
optimizer	
loss	
effective_loss_function	
activation	ReLU in GIN/MLP hidden layers; sigmoid/logit interpretation at binary output
train_ratio	
test_ratio	
balance_strategy	
balance_operation	
pos_weight	
epochs	
hidden_channels	
negative_ratio	
train_size	
validation_size	

9.7 Training Results

The following table records observed training and validation/test behavior per epoch. The current execution uses a short epoch budget to keep the end-to-end validation fast; article-grade runs can increase the epoch budget while reporting the same table.

No rows available.

9.8 Formula to Parameter Mapping

formula	meaning	values_in_run
---------	---------	---------------

$h_v^{\wedge}(k) = \text{MLP}^{\wedge}(k)((1 + \text{eps}^{\wedge}(k)) h_v^{\wedge}(k-1) + \sum_{u \in N(v)} h_u^{\wedge}(k-1))$	GIN layer update. Neighbor messages are summed and transformed by an MLP.	hidden_channels=
$z_G = \text{mean_pool}(\{h_v^{\wedge}(K)\})$	Subgraph embedding. Node embeddings are pooled into one vector.	embedding dimension=
$y_{\text{hat}} = \text{sigmoid}(\text{MLP}([z_{G1} z_{G2}]))$	Pair classifier. Two subgraph embeddings are concatenated and classified.	binary target: duplicate=1, not_duplicate=0
$L = -[y \log(\text{sigmoid}(z)) + (1-y) \log(1-\text{sigmoid}(z))]$	Configured training loss: .	optimizer=, learning_rate=, pos_weight=
$N_{\text{negative_sampled}} = N_{\text{positive}} * \text{negative_ratio}$	Selected imbalance strategy: .	operation=

10 Model Artifact

The trained pickle is copied into the report package and also referenced back to its original training location. For article reporting, GNN artifacts are interpreted as a detector family/cluster with scenario-specific artifact routing, not as one benchmark row per pickle.

field	value
original_model_path	
package_model_path	models/model.pkl
expected_input	normalized graph plus supervised pair table generated by the Scenario Materialization API

10.1 Model Family / Cluster Reporting

The benchmark compares detector families. VF2 and Node Match are deterministic families without pickle artifacts. GNN v1 and GNN v2 are reported as families with scenario-specific pickle routing. This keeps the paper readable: the number of benchmark rows is the number of detector families, while the number of pickle files is reported as artifact count. The routing table is mandatory evidence: a cluster can only be interpreted as complete when every evaluated scenario maps to an explicit pickle artifact.

model_family	artifact_count	scenario_count	coverage	reporting_rule
GNN TPC-DS v1 cluster	20	20	20/20	detector family with scenario-specific pickle routing
GNN GenAI SPEC v1 cluster	20	20	20/20	detector family with scenario-specific pickle routing
GNN GenAI SPEC Protocol rank 1	20	20	20/20	detector family with scenario-specific pickle routing
GNN GenAI SPEC Protocol rank 2	20	20	20/20	detector family with scenario-specific pickle routing
GNN GenAI SPEC Protocol rank 3	20	20	20/20	detector family with scenario-specific pickle routing

GNN GenAI SPEC Protocol rank 4	20	20	20/20	detector family with scenario-specific pickle routing
GNN GenAI SPEC Protocol rank 5	20	20	20/20	detector family with scenario-specific pickle routing
GNN Best-of-all cluster selector	20	20	20/20	post-hoc diagnostic selector; cite separately from fixed detector families

10.2 Scenario-Specific Artifact Routing

The PDF view uses artifact file names to keep the table readable. The exported CSV keeps the full absolute artifact paths for reproducibility.

model_ family	scenario	artifact_file	artifact_role	route_mode	route_source
GNN TPC- DS v1 cluster	graph_SOR2_ D1_seed42	graph_SOR2_ D1_seed42.pkl	scenario_ specific_pickle	scenario_specific	explicit scenario pickle map
GNN GenAI SPEC v1 cluster	graph_SOR2_ D1_seed42	GNN_tpcds_ genai_spec_ graph_SOR2_ D1_seed42.pkl	scenario_ specific_pickle	scenario_specific	explicit scenario pickle map
GNN GenAI SPEC Pro- tocol rank 1	graph_SOR2_ D1_seed42	GNN_tpcds_ genai_spec_ protocol_rank1_ graph_SOR2_ D1_seed42.pkl	scenario_ specific_pickle	scenario_specific	explicit scenario pickle map
GNN GenAI SPEC Pro- tocol rank 2	graph_SOR2_ D1_seed42	GNN_tpcds_ genai_spec_ protocol_rank2_ graph_SOR2_ D1_seed42.pkl	scenario_ specific_pickle	scenario_specific	explicit scenario pickle map
GNN GenAI SPEC Pro- tocol rank 3	graph_SOR2_ D1_seed42	GNN_tpcds_ genai_spec_ protocol_rank3_ graph_SOR2_ D1_seed42.pkl	scenario_ specific_pickle	scenario_specific	explicit scenario pickle map
GNN GenAI SPEC Pro- tocol rank 4	graph_SOR2_ D1_seed42	GNN_tpcds_ genai_spec_ protocol_rank4_ graph_SOR2_ D1_seed42.pkl	scenario_ specific_pickle	scenario_specific	explicit scenario pickle map
GNN GenAI SPEC Pro- tocol rank 5	graph_SOR2_ D1_seed42	GNN_tpcds_ genai_spec_ protocol_rank5_ graph_SOR2_ D1_seed42.pkl	scenario_ specific_pickle	scenario_specific	explicit scenario pickle map
GNN TPC- DS v1 cluster	graph_SOR2_ D2_seed42	graph_SOR2_ D2_seed42.pkl	scenario_ specific_pickle	scenario_specific	explicit scenario pickle map
GNN GenAI SPEC v1 cluster	graph_SOR2_ D2_seed42	GNN_tpcds_ genai_spec_ graph_SOR2_ D2_seed42.pkl	scenario_ specific_pickle	scenario_specific	explicit scenario pickle map
GNN GenAI SPEC Pro- tocol rank 1	graph_SOR2_ D2_seed42	GNN_tpcds_ genai_spec_ protocol_rank1_ graph_SOR2_ D2_seed42.pkl	scenario_ specific_pickle	scenario_specific	explicit scenario pickle map

GNN GenAI SPEC Pro- tocol rank 2	graph_SOR2_ D2_seed42	GNN_tpcds_ genai_spec_ protocol_rank2_ graph_SOR2_ D2_seed42.pkl	scenario_ specific_pickle	scenario_specific	explicit scenario pickle map
GNN GenAI SPEC Pro- tocol rank 3	graph_SOR2_ D2_seed42	GNN_tpcds_ genai_spec_ protocol_rank3_ graph_SOR2_ D2_seed42.pkl	scenario_ specific_pickle	scenario_specific	explicit scenario pickle map
GNN GenAI SPEC Pro- tocol rank 4	graph_SOR2_ D2_seed42	GNN_tpcds_ genai_spec_ protocol_rank4_ graph_SOR2_ D2_seed42.pkl	scenario_ specific_pickle	scenario_specific	explicit scenario pickle map
GNN GenAI SPEC Pro- tocol rank 5	graph_SOR2_ D2_seed42	GNN_tpcds_ genai_spec_ protocol_rank5_ graph_SOR2_ D2_seed42.pkl	scenario_ specific_pickle	scenario_specific	explicit scenario pickle map
GNN TPC- DS v1 cluster	graph_SOR2_ D3_seed42	graph_SOR2_ D3_seed42.pkl	scenario_ specific_pickle	scenario_specific	explicit scenario pickle map
GNN GenAI SPEC v1 cluster	graph_SOR2_ D3_seed42	GNN_tpcds_ genai_spec_ graph_SOR2_ D3_seed42.pkl	scenario_ specific_pickle	scenario_specific	explicit scenario pickle map
GNN GenAI SPEC Pro- tocol rank 1	graph_SOR2_ D3_seed42	GNN_tpcds_ genai_spec_ protocol_rank1_ graph_SOR2_ D3_seed42.pkl	scenario_ specific_pickle	scenario_specific	explicit scenario pickle map
GNN GenAI SPEC Pro- tocol rank 2	graph_SOR2_ D3_seed42	GNN_tpcds_ genai_spec_ protocol_rank2_ graph_SOR2_ D3_seed42.pkl	scenario_ specific_pickle	scenario_specific	explicit scenario pickle map
GNN GenAI SPEC Pro- tocol rank 3	graph_SOR2_ D3_seed42	GNN_tpcds_ genai_spec_ protocol_rank3_ graph_SOR2_ D3_seed42.pkl	scenario_ specific_pickle	scenario_specific	explicit scenario pickle map
GNN GenAI SPEC Pro- tocol rank 4	graph_SOR2_ D3_seed42	GNN_tpcds_ genai_spec_ protocol_rank4_ graph_SOR2_ D3_seed42.pkl	scenario_ specific_pickle	scenario_specific	explicit scenario pickle map
GNN GenAI SPEC Pro- tocol rank 5	graph_SOR2_ D3_seed42	GNN_tpcds_ genai_spec_ protocol_rank5_ graph_SOR2_ D3_seed42.pkl	scenario_ specific_pickle	scenario_specific	explicit scenario pickle map
GNN TPC- DS v1 cluster	graph_SOR2_ D4_seed42	graph_SOR2_ D4_seed42.pkl	scenario_ specific_pickle	scenario_specific	explicit scenario pickle map
GNN GenAI SPEC v1 cluster	graph_SOR2_ D4_seed42	GNN_tpcds_ genai_spec_ graph_SOR2_ D4_seed42.pkl	scenario_ specific_pickle	scenario_specific	explicit scenario pickle map
GNN GenAI SPEC Pro- tocol rank 1	graph_SOR2_ D4_seed42	GNN_tpcds_ genai_spec_ protocol_rank1_ graph_SOR2_ D4_seed42.pkl	scenario_ specific_pickle	scenario_specific	explicit scenario pickle map

GNN GenAI SPEC Pro- tocol rank 2	graph_SOR2_ D4_seed42	GNN_tpcds_ genai_spec_ protocol_rank2_ graph_SOR2_ D4_seed42.pkl	scenario_ specific_pickle	scenario_specific	explicit scenario pickle map
GNN GenAI SPEC Pro- tocol rank 3	graph_SOR2_ D4_seed42	GNN_tpcds_ genai_spec_ protocol_rank3_ graph_SOR2_ D4_seed42.pkl	scenario_ specific_pickle	scenario_specific	explicit scenario pickle map
GNN GenAI SPEC Pro- tocol rank 4	graph_SOR2_ D4_seed42	GNN_tpcds_ genai_spec_ protocol_rank4_ graph_SOR2_ D4_seed42.pkl	scenario_ specific_pickle	scenario_specific	explicit scenario pickle map
GNN GenAI SPEC Pro- tocol rank 5	graph_SOR2_ D4_seed42	GNN_tpcds_ genai_spec_ protocol_rank5_ graph_SOR2_ D4_seed42.pkl	scenario_ specific_pickle	scenario_specific	explicit scenario pickle map
GNN TPC- DS v1 cluster	graph_SOR2_ D5_seed42	graph_SOR2_ D5_seed42.pkl	scenario_ specific_pickle	scenario_specific	explicit scenario pickle map
GNN GenAI SPEC v1 cluster	graph_SOR2_ D5_seed42	GNN_tpcds_ genai_spec_ graph_SOR2_ D5_seed42.pkl	scenario_ specific_pickle	scenario_specific	explicit scenario pickle map

Only the first 30 rows are shown; the full table is exported as CSV.

11 Benchmark Metrics and Future Runs

Benchmark metrics are different from training metrics. Training metrics describe how the GNN behaved during supervised fitting. Benchmark metrics compare complete detector outputs against the selected ground truth and also measure runtime. Repetition medians are used because local execution time and stochastic training can vary between runs.

In Isomera v2, ET is the median detector runtime for the evaluated scenario. SF metrics are not copies of Accuracy or Jaccard: they divide the score by ET and scale by the number of evaluated candidate pairs, preserving the original Isomera interpretation of successful pair decisions per second. Detector-family summaries report scenario-level SF values aggregated across the selected benchmark scenarios.

Accuracy is retained as a diagnostic metric, but it is not the primary comparison metric for imbalanced duplicate detection. When most candidate pairs are negative, a detector can obtain high accuracy by correctly rejecting negatives while still failing to find duplicate pairs. For this reason, the report emphasizes SF-Jaccard: it uses Jaccard to focus on duplicate-pair overlap and then normalizes that overlap by execution time and evaluated pair volume.

11.1 Benchmark Metric Formulas

metric	formula	interpretation
Accuracy	$(TP + TN) / (TP + TN + FP + FN)$	Fraction of duplicate and non-duplicate pair decisions classified correctly.
Jaccard	$TP / (TP + FP + FN)$	Overlap between predicted duplicate pairs and validated duplicate pairs.

ET	$\text{median}(t_i), i = 1..runs$	Median wall-clock execution time for the detector on the scenario.
SF_accuracy	$\text{Accuracy} * N_pairs / ET$	Throughput-like rate of correct pair decisions per second.
SF_jaccard	$\text{Jaccard} * N_pairs / ET$	Jaccard-adjusted rate of successful duplicate-pair overlap per second.

11.2 Primary View: SF-Jaccard

The primary table places SF-Jaccard first and keeps Accuracy visible as a secondary diagnostic column. A high Accuracy with Jaccard equal to zero indicates that the detector is mostly benefiting from true negatives rather than detecting validated duplicate pairs.

algorithm	sf_jaccard	jaccard	ET	accuracy	sf_accuracy	N_pairs
VF2	2987.170696	0.071108	0.020127	0.63998	17634.003533	39695
Node Match	3522.146682	0.079101	0.020179	0.6853	17602.146628	39695
GNN TPC-DS v1 cluster	1186.392384	0.052612	0.025866	0.936037	21373.164634	39695
GNN GenAI SPEC v1 cluster	5005.201554	0.153409	0.0332	0.87613	25351.820589	39695
GNN GenAI SPEC Protocol rank 1	4702.327075	0.140115	0.03181	0.901826	28694.257933	39695
GNN GenAI SPEC Protocol rank 2	6185.156152	0.154068	0.03319	0.881597	26147.017658	39695
GNN GenAI SPEC Protocol rank 3	6078.918438	0.153409	0.023286	0.87613	28514.755418	39695
GNN GenAI SPEC Protocol rank 4	5571.160688	0.155142	0.029341	0.881194	26086.943007	39695
GNN GenAI SPEC Protocol rank 5	5124.503282	0.147343	0.031359	0.919965	27292.887697	39695
GNN Best-of-all cluster selector	7907.689842	0.152721	0.022447	0.880363	35026.528571	39695

11.3 Theory, Result, and Interpretation

This table connects the theoretical expectation of the detector family with the observed benchmark result. It is the paragraph-level bridge that should be used when writing the article discussion section.

algorithm	observed_result	interpretation	article_takeaway
VF2	SF-Jaccard=2987.17; Jaccard=0.071108; Accuracy=0.63998; ET=0.020127	Detected some duplicate overlap; compare against the best strategy using SF-Jaccard and ET.	SF-Jaccard is primary; accuracy is diagnostic because negatives dominate the candidate table.
Node Match	SF-Jaccard=3522.15; Jaccard=0.079101; Accuracy=0.6853; ET=0.020179	Detected some duplicate overlap; compare against the best strategy using SF-Jaccard and ET.	SF-Jaccard is primary; accuracy is diagnostic because negatives dominate the candidate table.
GNN TPC-DS v1 cluster	SF-Jaccard=1186.39; Jaccard=0.052612; Accuracy=0.936037; ET=0.025866	Detected some duplicate overlap; compare against the best strategy using SF-Jaccard and ET.	SF-Jaccard is primary; accuracy is diagnostic because negatives dominate the candidate table.

GNN GenAI SPEC v1 cluster	SF-Jaccard=5005.2; Jaccard=0.153409; Accuracy=0.87613; ET=0.0332	Detected some duplicate overlap; compare against the best strategy using SF-Jaccard and ET.	SF-Jaccard is primary; accuracy is diagnostic because negatives dominate the candidate table.
GNN GenAI SPEC Protocol rank 1	SF-Jaccard=4702.33; Jaccard=0.140115; Accuracy=0.901826; ET=0.03181	Detected some duplicate overlap; compare against the best strategy using SF-Jaccard and ET.	SF-Jaccard is primary; accuracy is diagnostic because negatives dominate the candidate table.
GNN GenAI SPEC Protocol rank 2	SF-Jaccard=6185.16; Jaccard=0.154068; Accuracy=0.881597; ET=0.03319	Detected some duplicate overlap; compare against the best strategy using SF-Jaccard and ET.	SF-Jaccard is primary; accuracy is diagnostic because negatives dominate the candidate table.
GNN GenAI SPEC Protocol rank 3	SF-Jaccard=6078.92; Jaccard=0.153409; Accuracy=0.87613; ET=0.023286	Detected some duplicate overlap; compare against the best strategy using SF-Jaccard and ET.	SF-Jaccard is primary; accuracy is diagnostic because negatives dominate the candidate table.
GNN GenAI SPEC Protocol rank 4	SF-Jaccard=5571.16; Jaccard=0.155142; Accuracy=0.881194; ET=0.029341	Detected some duplicate overlap; compare against the best strategy using SF-Jaccard and ET.	SF-Jaccard is primary; accuracy is diagnostic because negatives dominate the candidate table.
GNN GenAI SPEC Protocol rank 5	SF-Jaccard=5124.5; Jaccard=0.147343; Accuracy=0.919965; ET=0.031359	Detected some duplicate overlap; compare against the best strategy using SF-Jaccard and ET.	SF-Jaccard is primary; accuracy is diagnostic because negatives dominate the candidate table.
GNN Best-of-all cluster selector	SF-Jaccard=7907.69; Jaccard=0.152721; Accuracy=0.880363; ET=0.022447	Best primary metric in this run; use as the current article candidate configuration.	SF-Jaccard is primary; accuracy is diagnostic because negatives dominate the candidate table.

11.4 Full Benchmark Summary by Detector Family

algorithm	accuracy	jaccard	sf_jaccard	sf_accuracy	ET	N_pairs
VF2	0.63998	0.071108	2987.170696	17634.003533	0.020127	39695
Node Match	0.6853	0.079101	3522.146682	17602.146628	0.020179	39695
GNN TPC-DS v1 cluster	0.936037	0.052612	1186.392384	21373.164634	0.025866	39695
GNN GenAI SPEC v1 cluster	0.87613	0.153409	5005.201554	25351.820589	0.0332	39695
GNN GenAI SPEC Protocol rank 1	0.901826	0.140115	4702.327075	28694.257933	0.03181	39695
GNN GenAI SPEC Protocol rank 2	0.881597	0.154068	6185.156152	26147.017658	0.03319	39695
GNN GenAI SPEC Protocol rank 3	0.87613	0.153409	6078.918438	28514.755418	0.023286	39695
GNN GenAI SPEC Protocol rank 4	0.881194	0.155142	5571.160688	26086.943007	0.029341	39695
GNN GenAI SPEC Protocol rank 5	0.919965	0.147343	5124.503282	27292.887697	0.031359	39695
GNN Best-of-all cluster selector	0.880363	0.152721	7907.689842	35026.528571	0.022447	39695

11.5 Benchmark Results by Scenario

scenario	algorithm	accuracy	jaccard	sf_jaccard	sf_accuracy	ET	N_pairs	runs
graph__SOR2_D1__seed42	VF2	0.866667	0.333333	6839.36031	17782.336807	0.000731	15	10
graph__SOR2_D1__seed42	Node Match	0.933333	0.5	12550.116853	23426.884793	0.000598	15	10
graph__SOR2_D1__seed42	GNN TPC-DS v1 cluster	0.866667	0.333333	3060.872757	7958.269168	0.001634	15	10
graph__SOR2_D1__seed42	GNN GenAI SPEC v1 cluster	0.866667	0.333333	3341.175077	8687.055201	0.001496	15	10
graph__SOR2_D1__seed42	GNN GenAI SPEC Protocol rank 1	1.0	1.0	10268.84179	10268.84179	0.001461	15	10
graph__SOR2_D1__seed42	GNN GenAI SPEC Protocol rank 2	0.866667	0.333333	3447.334389	8963.069412	0.00145	15	10
graph__SOR2_D1__seed42	GNN GenAI SPEC Protocol rank 3	0.866667	0.333333	3421.972699	8897.129016	0.001461	15	10
graph__SOR2_D1__seed42	GNN GenAI SPEC Protocol rank 4	0.866667	0.333333	3426.513917	8908.936185	0.001459	15	10
graph__SOR2_D1__seed42	GNN GenAI SPEC Protocol rank 5	1.0	1.0	10412.451065	10412.451065	0.001441	15	10
graph__SOR2_D2__seed42	VF2	0.727273	0.1	1496.669402	10884.86838	0.00441	66	10
graph__SOR2_D2__seed42	Node Match	0.787879	0.125	1989.280077	12538.492604	0.004147	66	10
graph__SOR2_D2__seed42	GNN TPC-DS v1 cluster	0.954545	0.0	0.0	16207.699226	0.003887	66	10
graph__SOR2_D2__seed42	GNN GenAI SPEC v1 cluster	0.924242	0.285714	4968.099038	16071.047647	0.003796	66	10
graph__SOR2_D2__seed42	GNN GenAI SPEC Protocol rank 1	0.954545	0.0	0.0	16524.225931	0.003813	66	10
graph__SOR2_D2__seed42	GNN GenAI SPEC Protocol rank 2	0.924242	0.285714	5101.982884	16504.141602	0.003696	66	10
graph__SOR2_D2__seed42	GNN GenAI SPEC Protocol rank 3	0.924242	0.285714	5041.202696	16307.526905	0.003741	66	10
graph__SOR2_D2__seed42	GNN GenAI SPEC Protocol rank 4	0.924242	0.285714	5051.809808	16341.839304	0.003733	66	10

graph__ SOR2_D2__ seed42	GNN GenAI SPEC Pro- tocol rank 5	0.924242	0.285714	5110.971906	16533. 219726	0.00369	66	10
graph__ SOR2_D3__ seed42	VF2	0.75817	0.097561	1537.187739	11945. 857661	0.00971	153	10
graph__ SOR2_D3__ seed42	Node Match	0.75817	0.097561	1294.430198	10059. 330099	0.011532	153	10
graph__ SOR2_D3__ seed42	GNN TPC- DS v1 clus- ter	0.96732	0.0	0.0	8347.404546	0.01773	153	10
graph__ SOR2_D3__ seed42	GNN GenAI SPEC v1 cluster	0.869281	0.130435	3079.601313	20523. 966052	0.00648	153	10
graph__ SOR2_D3__ seed42	GNN GenAI SPEC Pro- tocol rank 1	0.869281	0.130435	2930.042834	19527. 235357	0.006811	153	10
graph__ SOR2_D3__ seed42	GNN GenAI SPEC Pro- tocol rank 2	0.869281	0.130435	2778.580994	18517.81974	0.007182	153	10
graph__ SOR2_D3__ seed42	GNN GenAI SPEC Pro- tocol rank 3	0.869281	0.130435	1344.496013	8960.37757	0.014843	153	10
graph__ SOR2_D3__ seed42	GNN GenAI SPEC Pro- tocol rank 4	0.869281	0.130435	2252.222553	15009.91022	0.008861	153	10
graph__ SOR2_D3__ seed42	GNN GenAI SPEC Pro- tocol rank 5	0.869281	0.130435	4848.621591	32313. 580496	0.004116	153	10
graph__ SOR2_D4__ seed42	VF2	0.721014	0.083333	3631.160608	31417. 433083	0.006334	276	10
graph__ SOR2_D4__ seed42	Node Match	0.721014	0.083333	3569.315809	30882. 341131	0.006444	276	10
graph__ SOR2_D4__ seed42	GNN TPC- DS v1 clus- ter	0.974638	0.0	0.0	47827. 715199	0.005624	276	10
graph__ SOR2_D4__ seed42	GNN GenAI SPEC v1 cluster	0.902174	0.129032	6312.129159	44133. 446539	0.005642	276	10
graph__ SOR2_D4__ seed42	GNN GenAI SPEC Pro- tocol rank 1	0.974638	0.0	0.0	44858. 409325	0.005997	276	10
graph__ SOR2_D4__ seed42	GNN GenAI SPEC Pro- tocol rank 2	0.902174	0.129032	3649.657463	25517. 849599	0.009758	276	10
graph__ SOR2_D4__ seed42	GNN GenAI SPEC Pro- tocol rank 3	0.902174	0.129032	2569.125984	17962. 937925	0.013862	276	10

graph__ SOR2_D4__ seed42	GNN GenAI SPEC Pro- tocol rank 4	0.902174	0.129032	3805.525725	26607. 656768	0.009358	276	10
graph__ SOR2_D4__ seed42	GNN GenAI SPEC Pro- tocol rank 5	0.974638	0.0	0.0	42382. 368027	0.006347	276	10
graph__ SOR2_D5__ seed42	VF2	0.712644	0.074074	2623.336312	25238.30452	0.012283	435	10
graph__ SOR2_D5__ seed42	Node Match	0.735632	0.08	2017.357216	18550. 411185	0.01725	435	10
graph__ SOR2_D5__ seed42	GNN TPC- DS v1 clus- ter	0.96092	0.0	0.0	22319. 248088	0.018728	435	10
graph__ SOR2_D5__ seed42	GNN GenAI SPEC v1 cluster	0.857471	0.074627	908.98264	10444. 315016	0.035713	435	10

Only the first 40 rows are shown; the full table is exported as CSV.

11.6 Per-Pickle Benchmark Results

This table links each GNN result to the exact pickle artifact used for that scenario. It is the table to cite when discussing model clusters, because each row is a concrete scenario–pickle–metric execution.

The PDF view uses artifact file names; the full absolute artifact paths are preserved in benchmark_pickle_results.csv.

scenario	algorithm	artifact__ file	artifact__ role	route__ mode	sf_jaccard	jaccard	ET	N_pairs
graph__ SOR2_D1__ seed42	GNN TPC- DS v1 clus- ter	graph__ SOR2_D1__ seed42.pkl	scenario__ specific__ pickle	scenario__ specific	3060.872757	0.333333	0.001634	15
graph__ SOR2_D1__ seed42	GNN GenAI SPEC v1 cluster	GNN__ tpcds__ genai__ spec__ graph__ SOR2_D1__ seed42.pkl	scenario__ specific__ pickle	scenario__ specific	3341.175077	0.333333	0.001496	15
graph__ SOR2_D1__ seed42	GNN GenAI SPEC Pro- tocol rank 1	GNN__ tpcds__ genai__ spec__ protocol__ rank1__ graph__ SOR2_D1__ seed42.pkl	scenario__ specific__ pickle	scenario__ specific	10268.84179	1.0	0.001461	15

graph__ SOR2_D1__ seed42	GNN GenAI SPEC Pro- tocol rank 2	GNN__ tpcds__ genai__ spec__ protocol__ rank2__ graph__ SOR2_D1__ seed42.pkl	scenario__ specific__ pickle	scenario__ specific	3447.334389	0.333333	0.00145	15
graph__ SOR2_D1__ seed42	GNN GenAI SPEC Pro- tocol rank 3	GNN__ tpcds__ genai__ spec__ protocol__ rank3__ graph__ SOR2_D1__ seed42.pkl	scenario__ specific__ pickle	scenario__ specific	3421.972699	0.333333	0.001461	15
graph__ SOR2_D1__ seed42	GNN GenAI SPEC Pro- tocol rank 4	GNN__ tpcds__ genai__ spec__ protocol__ rank4__ graph__ SOR2_D1__ seed42.pkl	scenario__ specific__ pickle	scenario__ specific	3426.513917	0.333333	0.001459	15
graph__ SOR2_D1__ seed42	GNN GenAI SPEC Pro- tocol rank 5	GNN__ tpcds__ genai__ spec__ protocol__ rank5__ graph__ SOR2_D1__ seed42.pkl	scenario__ specific__ pickle	scenario__ specific	10412. 451065	1.0	0.001441	15
graph__ SOR2_D2__ seed42	GNN TPC- DS v1 clus- ter	graph__ SOR2_D2__ seed42.pkl	scenario__ specific__ pickle	scenario__ specific	0.0	0.0	0.003887	66
graph__ SOR2_D2__ seed42	GNN GenAI SPEC v1 cluster	GNN__ tpcds__ genai__ spec__ graph__ SOR2_D2__ seed42.pkl	scenario__ specific__ pickle	scenario__ specific	4968.099038	0.285714	0.003796	66
graph__ SOR2_D2__ seed42	GNN GenAI SPEC Pro- tocol rank 1	GNN__ tpcds__ genai__ spec__ protocol__ rank1__ graph__ SOR2_D2__ seed42.pkl	scenario__ specific__ pickle	scenario__ specific	0.0	0.0	0.003813	66
graph__ SOR2_D2__ seed42	GNN GenAI SPEC Pro- tocol rank 2	GNN__ tpcds__ genai__ spec__ protocol__ rank2__ graph__ SOR2_D2__ seed42.pkl	scenario__ specific__ pickle	scenario__ specific	5101.982884	0.285714	0.003696	66

graph__ SOR2_D2__ seed42	GNN GenAI SPEC Pro- tocol rank 3	GNN__ tpcds__ genai__ spec__ protocol__ rank3__ graph__ SOR2_D2__ seed42.pkl	scenario__ specific__ pickle	scenario__ specific	5041.202696	0.285714	0.003741	66
graph__ SOR2_D2__ seed42	GNN GenAI SPEC Pro- tocol rank 4	GNN__ tpcds__ genai__ spec__ protocol__ rank4__ graph__ SOR2_D2__ seed42.pkl	scenario__ specific__ pickle	scenario__ specific	5051.809808	0.285714	0.003733	66
graph__ SOR2_D2__ seed42	GNN GenAI SPEC Pro- tocol rank 5	GNN__ tpcds__ genai__ spec__ protocol__ rank5__ graph__ SOR2_D2__ seed42.pkl	scenario__ specific__ pickle	scenario__ specific	5110.971906	0.285714	0.00369	66
graph__ SOR2_D3__ seed42	GNN TPC- DS v1 clus- ter	graph__ SOR2_D3__ seed42.pkl	scenario__ specific__ pickle	scenario__ specific	0.0	0.0	0.01773	153
graph__ SOR2_D3__ seed42	GNN GenAI SPEC v1 cluster	GNN__ tpcds__ genai__ spec__ graph__ SOR2_D3__ seed42.pkl	scenario__ specific__ pickle	scenario__ specific	3079.601313	0.130435	0.00648	153
graph__ SOR2_D3__ seed42	GNN GenAI SPEC Pro- tocol rank 1	GNN__ tpcds__ genai__ spec__ protocol__ rank1__ graph__ SOR2_D3__ seed42.pkl	scenario__ specific__ pickle	scenario__ specific	2930.042834	0.130435	0.006811	153
graph__ SOR2_D3__ seed42	GNN GenAI SPEC Pro- tocol rank 2	GNN__ tpcds__ genai__ spec__ protocol__ rank2__ graph__ SOR2_D3__ seed42.pkl	scenario__ specific__ pickle	scenario__ specific	2778.580994	0.130435	0.007182	153
graph__ SOR2_D3__ seed42	GNN GenAI SPEC Pro- tocol rank 3	GNN__ tpcds__ genai__ spec__ protocol__ rank3__ graph__ SOR2_D3__ seed42.pkl	scenario__ specific__ pickle	scenario__ specific	1344.496013	0.130435	0.014843	153

graph__ SOR2_D3__ seed42	GNN GenAI SPEC Pro- tocol rank 4	GNN__ tpcds__ genai__ spec__ protocol__ rank4__ graph__ SOR2_D3__ seed42.pkl	scenario__ specific__ pickle	scenario__ specific	2252.222553	0.130435	0.008861	153
graph__ SOR2_D3__ seed42	GNN GenAI SPEC Pro- tocol rank 5	GNN__ tpcds__ genai__ spec__ protocol__ rank5__ graph__ SOR2_D3__ seed42.pkl	scenario__ specific__ pickle	scenario__ specific	4848.621591	0.130435	0.004116	153
graph__ SOR2_D4__ seed42	GNN TPC- DS v1 clus- ter	graph__ SOR2_D4__ seed42.pkl	scenario__ specific__ pickle	scenario__ specific	0.0	0.0	0.005624	276
graph__ SOR2_D4__ seed42	GNN GenAI SPEC v1 cluster	GNN__ tpcds__ genai__ spec__ graph__ SOR2_D4__ seed42.pkl	scenario__ specific__ pickle	scenario__ specific	6312.129159	0.129032	0.005642	276
graph__ SOR2_D4__ seed42	GNN GenAI SPEC Pro- tocol rank 1	GNN__ tpcds__ genai__ spec__ protocol__ rank1__ graph__ SOR2_D4__ seed42.pkl	scenario__ specific__ pickle	scenario__ specific	0.0	0.0	0.005997	276
graph__ SOR2_D4__ seed42	GNN GenAI SPEC Pro- tocol rank 2	GNN__ tpcds__ genai__ spec__ protocol__ rank2__ graph__ SOR2_D4__ seed42.pkl	scenario__ specific__ pickle	scenario__ specific	3649.657463	0.129032	0.009758	276
graph__ SOR2_D4__ seed42	GNN GenAI SPEC Pro- tocol rank 3	GNN__ tpcds__ genai__ spec__ protocol__ rank3__ graph__ SOR2_D4__ seed42.pkl	scenario__ specific__ pickle	scenario__ specific	2569.125984	0.129032	0.013862	276
graph__ SOR2_D4__ seed42	GNN GenAI SPEC Pro- tocol rank 4	GNN__ tpcds__ genai__ spec__ protocol__ rank4__ graph__ SOR2_D4__ seed42.pkl	scenario__ specific__ pickle	scenario__ specific	3805.525725	0.129032	0.009358	276

graph__ SOR2_D4__ seed42	GNN GenAI SPEC Pro- tocol rank 5	GNN__ tpcds__ genai__ spec__ protocol__ rank5__ graph__ SOR2_D4__ seed42.pkl	scenario__ specific__ pickle	scenario__ specific	0.0	0.0	0.006347	276
graph__ SOR2_D5__ seed42	GNN TPC- DS v1 clus- ter	graph__ SOR2_D5__ seed42.pkl	scenario__ specific__ pickle	scenario__ specific	0.0	0.0	0.018728	435
graph__ SOR2_D5__ seed42	GNN GenAI SPEC v1 cluster	GNN__ tpcds__ genai__ spec__ graph__ SOR2_D5__ seed42.pkl	scenario__ specific__ pickle	scenario__ specific	908.98264	0.074627	0.035713	435
graph__ SOR2_D5__ seed42	GNN GenAI SPEC Pro- tocol rank 1	GNN__ tpcds__ genai__ spec__ protocol__ rank1__ graph__ SOR2_D5__ seed42.pkl	scenario__ specific__ pickle	scenario__ specific	1229.712899	0.042553	0.015053	435
graph__ SOR2_D5__ seed42	GNN GenAI SPEC Pro- tocol rank 2	GNN__ tpcds__ genai__ spec__ protocol__ rank2__ graph__ SOR2_D5__ seed42.pkl	scenario__ specific__ pickle	scenario__ specific	883.369959	0.074627	0.036749	435
graph__ SOR2_D5__ seed42	GNN GenAI SPEC Pro- tocol rank 3	GNN__ tpcds__ genai__ spec__ protocol__ rank3__ graph__ SOR2_D5__ seed42.pkl	scenario__ specific__ pickle	scenario__ specific	3128.387125	0.074627	0.010377	435
graph__ SOR2_D5__ seed42	GNN GenAI SPEC Pro- tocol rank 4	GNN__ tpcds__ genai__ spec__ protocol__ rank4__ graph__ SOR2_D5__ seed42.pkl	scenario__ specific__ pickle	scenario__ specific	3227.829677	0.074627	0.010057	435
graph__ SOR2_D5__ seed42	GNN GenAI SPEC Pro- tocol rank 5	GNN__ tpcds__ genai__ spec__ protocol__ rank5__ graph__ SOR2_D5__ seed42.pkl	scenario__ specific__ pickle	scenario__ specific	0.0	0.0	0.016278	435

graph__ SOR4_D1__ seed42	GNN TPC- DS v1 cluster	graph__ SOR4_D1__ seed42.pkl	scenario__ specific__ pickle	scenario__ specific	13453. 068383	0.75	0.003679	66
graph__ SOR4_D1__ seed42	GNN GenAI SPEC v1 cluster	GNN__ tpcds__ genai__ spec__ graph__ SOR4_D1__ seed42.pkl	scenario__ specific__ pickle	scenario__ specific	4633.755671	0.75	0.010682	66
graph__ SOR4_D1__ seed42	GNN GenAI SPEC Pro- tocol rank 1	GNN__ tpcds__ genai__ spec__ protocol__ rank1__ graph__ SOR4_D1__ seed42.pkl	scenario__ specific__ pickle	scenario__ specific	20064. 685328	0.75	0.002467	66
graph__ SOR4_D1__ seed42	GNN GenAI SPEC Pro- tocol rank 2	GNN__ tpcds__ genai__ spec__ protocol__ rank2__ graph__ SOR4_D1__ seed42.pkl	scenario__ specific__ pickle	scenario__ specific	26520. 523448	0.75	0.001866	66
graph__ SOR4_D1__ seed42	GNN GenAI SPEC Pro- tocol rank 3	GNN__ tpcds__ genai__ spec__ protocol__ rank3__ graph__ SOR4_D1__ seed42.pkl	scenario__ specific__ pickle	scenario__ specific	16927. 055404	0.75	0.002924	66

Only the first 40 rows are shown; the full table is exported as CSV.

11.7 Benchmark Figures

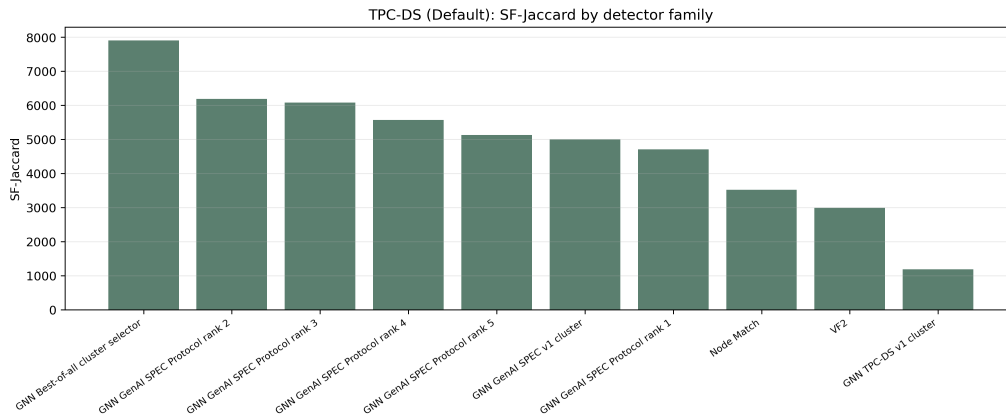


Figure 2: Primary metric: SF-Jaccard by detector family.

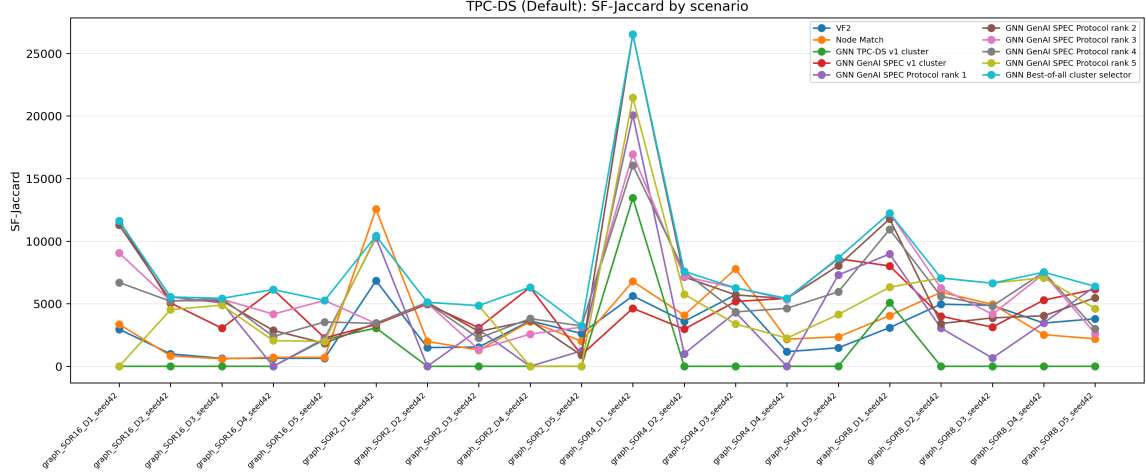


Figure 3: SF-Jaccard line plot by scenario and detector family.

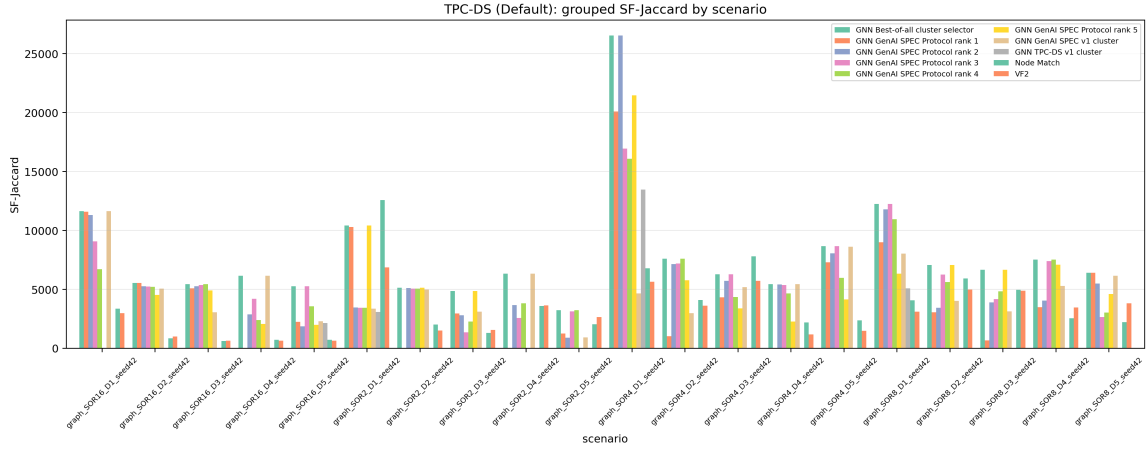


Figure 4: SF-Jaccard grouped bars by scenario and detector family.

12 Reproducibility Package

The accompanying zip package contains the LaTeX source, compiled PDF when available, source JSON, lineage and adjacency figures, source GML, labels, manifest, exported tables, model pickle, model metadata, and benchmark references.

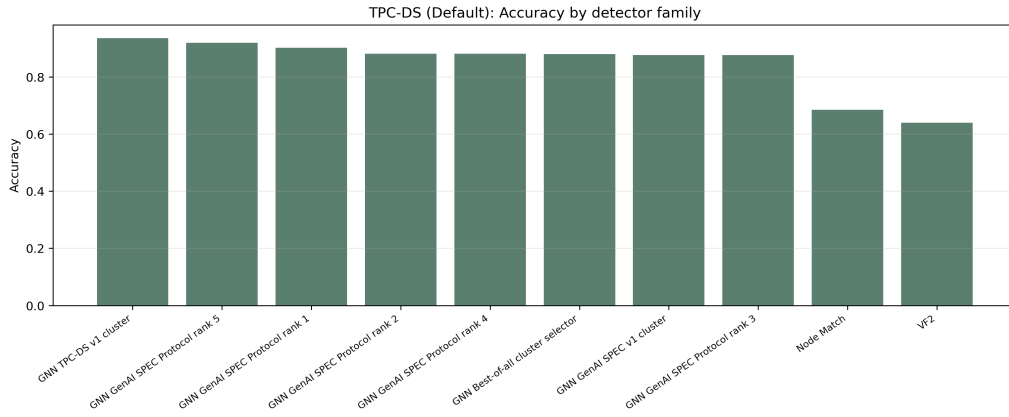


Figure 5: Benchmark accuracy by detector family.

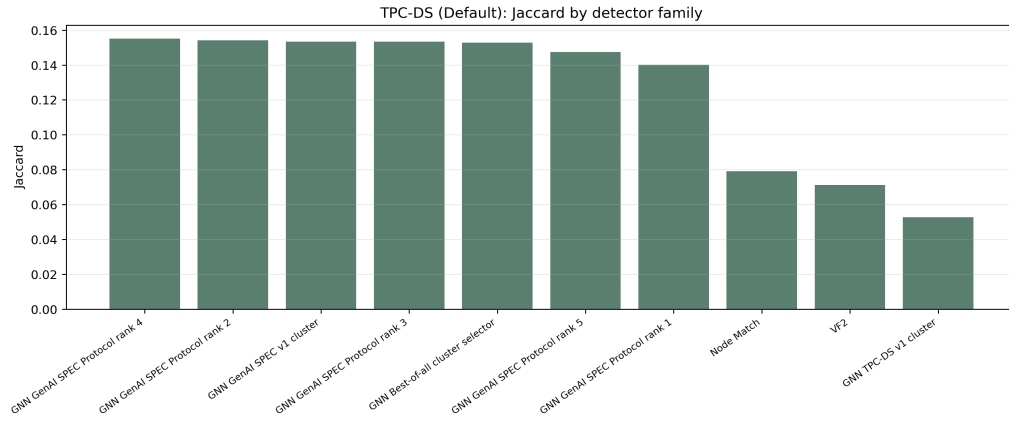


Figure 6: Benchmark Jaccard score by detector family.

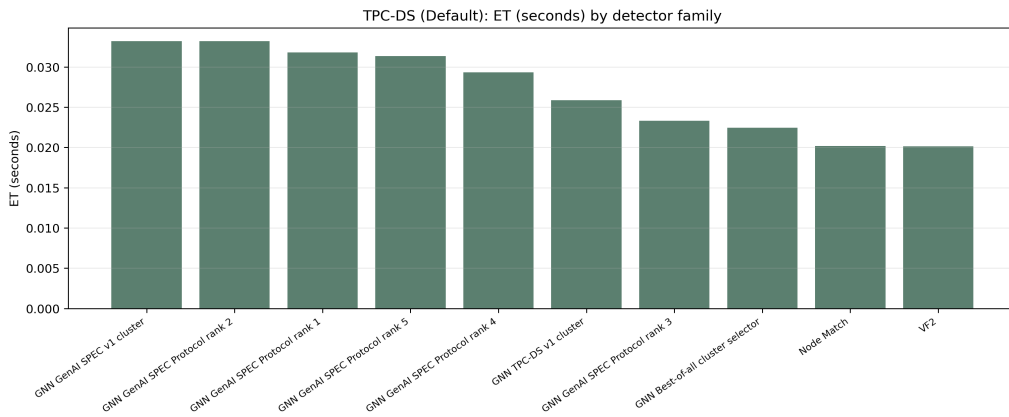


Figure 7: Benchmark ET by detector family.